

PROGRESSIVE BANKING APPLICATION

Detailed Project Description for Agile, Java Full Stack, Spring Boot & DevOps Training

1. Project Overview

The bank wants to develop a Digital Banking Application that enables customers to perform basic banking operations through a secure digital platform. The application will initially focus on core banking operations such as account creation, balance inquiry, deposits, withdrawals, fund transfers, and transaction history.

The project is intentionally designed as a progressive project. Trainees should not attempt to build the complete system in one step. The application should evolve through multiple phases, beginning with core Java business logic and gradually introducing layered architecture, database persistence, Spring Boot REST APIs, authentication, Spring Security, JWT, testing, Docker, CI/CD, and Kubernetes.

The project description is written as a business requirement so that trainees can independently derive Epics, Features, User Stories, Acceptance Criteria, Negative Scenarios, Tasks, and technical implementation steps.

2. Business Objective

- Provide customers with convenient access to common banking operations.
- Reduce dependency on branch-based banking for routine transactions.
- Maintain accurate account balances and transaction records.
- Ensure invalid or unauthorized banking operations are rejected.
- Provide a foundation that can be progressively enhanced into a production-style application.
- Use the project as a common case study across Agile, Java, Spring Boot, database, security, testing, and DevOps training.

3. Scope of the Initial Application

The initial business scope includes the following capabilities:

- Customer and account creation
- Account search and account details
- Balance inquiry
- Deposit money
- Withdraw money
- Transfer money between accounts
- Transaction history
- Basic validation and exception handling
- Basic customer and administrator roles

4. Users and Roles

Role	Description	Major Responsibilities
Customer	A registered bank customer.	Manage own account, check balance, deposit, withdraw, transfer money, view transactions.
Bank Administrator	An authorized bank employee.	View/search accounts, review transactions, activate/deactivate accounts.

5. Customer Account Management

The system should maintain customer and account information. A customer may have one or more accounts depending on the business rules introduced in later phases.

- Customer ID
- Customer name
- Email address
- Mobile number
- Address
- Account number
- Account type
- Initial deposit
- Account status
- Account creation date

The system must generate a unique account number. An account should have a status such as ACTIVE, INACTIVE, or BLOCKED. Only ACTIVE accounts should be allowed to perform financial transactions.

6. Account Types

- Savings Account – intended for normal customer banking operations.
- Current Account – intended for business/current-account style transactions.

For the initial implementation, both account types may follow the same transaction rules. Additional account-specific rules can be introduced later as an extension exercise.

7. Balance Inquiry

A customer should be able to retrieve the current balance of an active account. The balance should reflect all successfully completed deposits, withdrawals, and transfers.

Example:

Account	Number:	10010001
Customer:	Ravi	Kumar
Account	Type:	Savings
Balance:		₹50,000
Status: ACTIVE		

8. Deposit Money

A customer should be able to deposit money into an active account. The system must validate that the account exists, the account is active, and the deposit amount is greater than zero.

Example:

Previous	Balance:	₹50,000
Deposit:		₹5,000
New Balance: ₹55,000		

A successful deposit must create a transaction record.

9. Withdraw Money

A customer should be able to withdraw money from an active account. Before processing the withdrawal, the system must verify that the amount is valid and that sufficient available balance exists.

Example:

Available	Balance:	₹50,000
Withdrawal:		₹10,000
New Balance: ₹40,000		

If the withdrawal amount exceeds the available balance, the system must reject the transaction and must not change the account balance.

10. Fund Transfer

A customer should be able to transfer money from one account to another account within the banking application. The source and destination accounts must be valid and active.

Before processing a transfer, the system should validate:

1. Source account exists.
2. Destination account exists.
3. Source account is active.
4. Destination account is active.
5. Source and destination accounts are different.
6. Transfer amount is greater than zero.
7. Source account has sufficient available balance.

When the transfer succeeds, the source account must be debited and the destination account must be credited. A unique transaction reference must be generated, and the transaction details must be recorded.

11. Transaction Management

Every financial operation must produce an auditable transaction record.

- Transaction ID
- Account number
- Transaction type
- Amount
- Transaction date and time
- Transaction status
- Description/reference

Initial transaction types:

- DEPOSIT
- WITHDRAWAL
- TRANSFER

Initial transaction statuses:

- SUCCESS
- FAILED

A later phase may introduce PENDING for asynchronous or externally dependent transactions.

12. Transaction History

Customers should be able to view transactions associated with their accounts. The most recent transactions should be displayed first.

Date	Type	Amount	Status
28-Sep-2026	DEPOSIT	₹10,000	SUCCESS
28-Sep-2026	WITHDRAWAL	₹2,000	SUCCESS
29-Sep-2026	TRANSFER	₹5,000	SUCCESS

Later enhancements may include filtering by date range, transaction type, amount, and status.

13. Validation Requirements

Account Creation

- Customer name cannot be empty.
- Email and mobile number must satisfy the defined validation rules.
- Account type must be supported.
- Account number must be unique.

Deposit

- Account must exist.
- Account must be ACTIVE.
- Amount must be greater than zero.

Withdrawal

- Account must exist.
- Account must be ACTIVE.
- Amount must be greater than zero.
- Sufficient balance must be available.

Transfer

- Source account must exist.
- Destination account must exist.
- Source and destination must be different.
- Both accounts must be ACTIVE.
- Amount must be greater than zero.
- Source account must have sufficient balance.

14. Exception Handling

The application should handle expected business failures using meaningful exceptions and responses.

- AccountNotFoundException
- InsufficientBalanceException
- InvalidAmountException
- AccountInactiveException
- DuplicateAccountException
- InvalidTransactionException

Example error message:

Transaction Failed: Insufficient balance in source account.

15. Core Business Rules

8. Every account must have a unique account number.
9. Only ACTIVE accounts can perform financial transactions.
10. Deposit, withdrawal, and transfer amounts must be greater than zero.
11. An account balance must never become negative.
12. A customer cannot transfer money to the same account.
13. Both source and destination accounts must exist for a transfer.
14. A successful transaction must update the relevant account balance.
15. Every financial transaction must have a unique transaction ID.
16. A failed transaction must not incorrectly change an account balance.
17. Transaction history must be retained for financial operations.

16. Progressive Development Approach

The project should be developed in increments. Each phase should produce a working version of the application. Trainees should avoid rewriting the entire project for each phase; instead, they should extend the existing implementation.

Phase	Expected Work	Outcome
Phase 1 – Core Java	Create a console-based banking application. Implement account creation, account search, balance inquiry, deposit, withdrawal, and transfer using Java classes, objects, collections, interfaces, and exception handling.	Working banking business logic.
Phase 2 – Layered Architecture	Separate the application into Model/Entity, Repository, Service, and Controller-style layers. Introduce interfaces where appropriate and isolate business logic from input/output handling.	Maintainable layered application.
Phase 3 – Database Integration	Replace in-memory collections with a relational database such as MySQL, Oracle, or PostgreSQL. Introduce CUSTOMER, ACCOUNT, and TRANSACTION data structures and relationships.	Persistent banking data.
Phase 4 – Spring Boot REST API	Expose banking operations through REST APIs. Implement appropriate HTTP methods, request/response models, validation, status codes, and centralized exception handling.	API-based banking application.
Phase 5 – Authentication	Introduce customer registration/login and restrict access to customer-specific operations.	Authenticated application.
Phase 6 – Spring Security	Implement password protection, authentication, authorization, role-based access control, session/security policies, and endpoint protection.	Secured application.
Phase 7 – JWT	Introduce stateless JWT authentication for REST APIs. Validate tokens on protected requests and enforce authorization rules.	JWT-secured REST API.
Phase 8 – Testing	Add unit, integration, and API tests. Cover both successful and failure scenarios.	Tested application.
Phase 9 – Docker	Create a Dockerfile and containerize the application. Later introduce Docker Compose for the application and database.	Containerized application.

Phase 10 – CI/CD	Create a pipeline that checks out source code, builds the application, runs tests, creates a Docker image, and publishes/deploys it.	Automated delivery pipeline.
Phase 11 – Kubernetes	Deploy the containerized application to Kubernetes. Introduce Deployment, Service, ConfigMap/Secret concepts, scaling, and health checks.	Container-orchestrated application.

17. Suggested REST API Scope

Method	Endpoint	Purpose
POST	/api/accounts	Create account
GET	/api/accounts/{accountNumber}	Get account details
GET	/api/accounts/{accountNumber}/balance	Get balance
POST	/api/accounts/{accountNumber}/deposit	Deposit money
POST	/api/accounts/{accountNumber}/withdraw	Withdraw money
POST	/api/transfers	Transfer money
GET	/api/accounts/{accountNumber}/transactions	Get transaction history

18. Suggested Data Model

A basic relational design can start with the following entities:

- CUSTOMER – stores customer information.
- ACCOUNT – stores account number, type, status, and balance.
- TRANSACTION – stores deposits, withdrawals, and transfers.

The design can later be expanded with USER, ROLE, BENEFICIARY, LOGIN_AUDIT, NOTIFICATION, and SERVICE_REQUEST entities as security and functional requirements grow.

19. Security Expectations for Later Phases

- Customers must only access their own permitted account information.
- Passwords must never be stored as plain text.
- Sensitive APIs must require authentication.
- Authorization should be based on roles and permissions.
- Financial operations should be audited.
- Sensitive configuration such as database credentials and JWT secrets should not be hard-coded.
- Communication should use HTTPS in a production-style deployment.

20. Testing Expectations

Trainees should test both positive and negative scenarios.

- Create a valid account.
- Attempt to create a duplicate account where applicable.
- Deposit a valid amount.

- Attempt a zero or negative deposit.
- Withdraw within available balance.
- Attempt withdrawal exceeding balance.
- Transfer between two valid accounts.
- Attempt transfer to an invalid account.
- Attempt transfer with insufficient balance.
- Attempt transfer to the same account.
- Attempt transaction on an inactive account.
- Verify that successful transactions update balances correctly.
- Verify that failed transactions do not incorrectly update balances.
- Verify transaction history after each successful operation.

21. Agile Training Exercise

Trainees should receive this project description before implementation and be asked to analyze the business requirements themselves.

18. Identify the major Epics.
19. Identify Features under each Epic.
20. Write User Stories for the identified features.
21. Define Acceptance Criteria using Given/When/Then.
22. Identify positive and negative scenarios.
23. Extract Business Rules.
24. Break User Stories into development tasks.
25. Identify dependencies between stories.
26. Prioritize stories for an initial release.
27. Estimate stories using the team's chosen estimation technique.

22. Suggested Initial Release

The first release should remain intentionally small. A suggested scope is account creation, account lookup, balance inquiry, deposit, withdrawal, and transaction history. Fund transfer can be added as the next increment after the basic account operations are stable.

The goal is to ensure that each increment is demonstrable and testable before the next set of requirements is introduced.

23. Final Target Architecture

At the end of the progressive project, trainees should be able to evolve the application toward the following architecture:

Client → REST APIs → Spring Security/JWT → Controllers → Services → Spring Data JPA → Database

The delivery path can then evolve into:

24. Expected Learning Outcomes

- Understand how a real-world business requirement can be decomposed into Agile work items.
- Convert business requirements into Epics, Features, User Stories, and Acceptance Criteria.
- Implement core banking business rules using Java.
- Apply layered architecture and clean separation of responsibilities.
- Persist banking data using a relational database.
- Build production-style REST APIs with Spring Boot.
- Implement validation and exception handling.
- Secure APIs using Spring Security and JWT.
- Write unit, integration, and API tests.
- Containerize the application using Docker.
- Build a CI/CD pipeline.
- Deploy and operate the application using Kubernetes.

25. Important Note for Trainees

This is a training project and is not intended to represent a complete production banking system. Real banking systems require significantly stronger controls, regulatory compliance, fraud detection, high availability, disaster recovery, encryption/key management, transaction integrity mechanisms, observability, audit controls, and integration with regulated banking networks.